

Data-collection session guide

Operator runbook for recording a driver-attention dataset in the lab using:

- ****Jetson Orin Nano Super + ArduCam IMX477**** (sensor 0) – captures the driver-facing video.
- ****Lab PC running Assetto Corsa**** – provides the driving simulator on a separate display.

Both machines run in parallel during a session. Items marked `_(TODO: ...)_` are placeholders to fill in once the procedure is locked in.

0. Before the session

- [] Camera is mounted in its rig and aimed at the driver seat. Adjust so the driver's whole face is in frame at typical posture.
- [] Jetson is plugged into power and the display.
- [] PC is plugged in, monitor on, wheel/pedals connected.
- [] `_(TODO: confirm participant consent / IRB paperwork if required.)_`

1. Power up the Jetson and start the recorder

1. Power on the Jetson. Wait for the desktop.
2. Open a terminal.
3. Launch the recorder:

```
``bash
cd ~/Meng/driver-attn-monitor      # or wherever the repo lives on
this Jetson
./tools/start_recording.sh
````
```

This activates the ``dms-infer`` conda env and opens the **\*\*DMS Recorder\*\*** window – live camera preview on the left, status panel + buttons on the right.

4. The right panel shows ``Status: IDLE``. **\*\*Don't click Start yet\*\*** – wait until the simulator is ready.

If the preview is black or the script errors out:

- Re-run the camera-sensor selection step (``sudo /opt/nvidia/jetson-io/jetson-io.py`` → IMX477) and reboot. This setting sometimes resets after L4T updates – see ``docs/log.txt``.

---

### ## 2. Power up the PC and start Assetto Corsa

1. Power on the PC. Log in.
2. Launch Assetto Corsa `_(TODO: via Steam / desktop shortcut – confirm which.)_`
3. Set up a session with the standard configuration:

- **\*\*Mode:\*\*** \_(TODO: Practice / Hotlap / Race – pick one for consistency across sessions.)\_
- **\*\*Track:\*\*** \_(TODO: e.g. "Silverstone GP", "Monza".)\_
- **\*\*Track layout / config:\*\*** \_(TODO.)\_
- **\*\*Car:\*\*** \_(TODO: e.g. "Lotus Exos 125".)\_
- **\*\*Time of day:\*\*** \_(TODO.)\_
- **\*\*Weather:\*\*** \_(TODO.)\_
- **\*\*AI opponents:\*\*** \_(TODO: number, difficulty.)\_
- **\*\*Assists:\*\*** \_(TODO: TC/ABS/auto-clutch settings.)\_

4. Load into the car. Pause on the grid; don't start driving yet.

---

### ## 3. Start the recording

1. Driver gets into position at the wheel.
2. On the Jetson recorder window, click **\*\*Start Recording\*\***. The status flips to `● REC` in red, the elapsed counter starts, a red dot appears on the preview, and the terminal prints `[REC] start -> recordings/rec\_<timestamp>.mp4`.
3. On the PC, unpause Assetto Corsa and start driving.

Treat the recording as running from the Start click onwards; what happens on the PC is whatever the session plan calls for.

---

### ## 4. During the session

- The driver does \_(TODO: laps / scenario / event sequence – describe what the participant is supposed to do.)\_
- Keep an eye on the Jetson recorder from a non-distracting angle if you need to confirm the `● REC` status is still on and the elapsed timer is still counting.
- \_(TODO: any planned interruptions, distraction prompts, drowsiness cues?)\_

---

### ## 5. End the session

1. On the Jetson recorder, click **\*\*Stop Recording\*\***. The status flips back to `IDLE`, the file name moves down to "Last saved", and the terminal prints `[REC] stop -> recordings/rec\_<timestamp>.mp4`.
2. Click **\*\*Quit\*\*** (or close the window) to exit the recorder.
3. Stop / exit Assetto Corsa on the PC.

The MP4 is saved at:

```
\\
<repo>/recordings/rec_YYYYMMDD_HHMMSS.mp4
\\
```

`recordings/` is gitignored – files stay local to the Jetson.

---

## ## 6. After the session

- [ ] Log session metadata: `_`(TODO: where? spreadsheet / text file / per-recording sidecar? Suggested fields: date, participant ID, track, car, AI/weather settings, scenario, anomalies.)`_`
- [ ] Back up the file:
  - `_`(TODO: destination – external drive / lab NAS / cloud bucket?)`_`
- [ ] Verify the file plays end-to-end before clearing the Jetson SSD.

---

## ## Notes

- The recorder is a Tkinter app. Tkinter ships with Python, but ``tools/record.py`` also uses `**Pillow**` to bridge the OpenCV frame into the Tk preview – make sure ``pillow`` is in your ``dms-infer`` env (it is in ``inference/environment.yml``). If the app crashes with ``ModuleNotFoundError: PIL``, run ``pip install pillow`` inside the env.
- The recorder uses ``cv2.VideoWriter`` with the ``mp4v`` codec, encoded on the Jetson CPU. At 1280×720 / 30 fps this should hold realtime on Orin Nano. If you see dropped frames, the next step is a GStreamer sink with ``nv4l2h264enc`` (hardware encoder).
- All camera parameters (resolution, framerate, sensor id, flip) come from ``config.yaml`` – edit there, not in ``tools/record.py``.